

tf.compat.v1.make_template Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/make_template

Given an arbitrary function, wrap it so that it does variable sharing.

Function Signatures

```
tf.compat.v1.make_template( name_, func_,  
create_scope_now_=False, unique_name_=None,  
custom_getter_=None, **kwargs )
```

Code Examples

```
tf.compat.v1.make_template(  
name_,  
func_,  
create_scope_now_=False,  
unique_name_=None,  
custom_getter_=None,  
**kwargs  
)
```

```
tf.compat.v1.make_template(  
    name_,  
    func_,  
    create_scope_now_=False,  
    unique_name_=None,  
    custom_getter_=None,  
    **kwargs  
)
```

```
def my_op(x, scalar_name):  
    var1 = tf.compat.v1.get_variable(scalar_name,  
    shape=[],  
    initializer=tf.compat.v1.constant_initializer(1))  
    return x * var1  
    scale_by_y = tf.compat.v1.make_template('scale_by_y', my_op,  
    scalar_name='y')  
    z = scale_by_y(input1)  
    w = scale_by_y(input2)
```

```
def my_op(x, scalar_name):  
    var1 = tf.compat.v1.get_variable(scalar_name,  
    shape=[],  
    initializer=tf.compat.v1.constant_initializer(1))  
    return x * var1  
    scale_by_y = tf.compat.v1.make_template('scale_by_y', my_op,  
    scalar_name='y')  
    z = scale_by_y(input1)  
    w = scale_by_y(input2)
```

```

def my_op(x, scalar_name):
    var1 = tf.compat.v1.get_variable(scalar_name,
    shape=[],
    initializer=tf.compat.v1.constant_initializer(1))
    return x * var1
    with tf.compat.v1.variable_scope('scope') as vs:
        scale_by_y = tf.compat.v1.make_template('scale_by_y', my_op,
        scalar_name='y')
        z = scale_by_y(input1)
        w = scale_by_y(input2)
        # Creates a template that reuses the variables above.
        with tf.compat.v1.variable_scope(vs, reuse=True):
            scale_by_y2 = tf.compat.v1.make_template('scale_by_y', my_op,
            scalar_name='y')
            z2 = scale_by_y2(input1)
            w2 = scale_by_y2(input2)

```

```

def my_op(x, scalar_name):
    var1 = tf.compat.v1.get_variable(scalar_name,
    shape=[],
    initializer=tf.compat.v1.constant_initializer(1))
    return x * var1
    with tf.compat.v1.variable_scope('scope') as vs:
        scale_by_y = tf.compat.v1.make_template('scale_by_y', my_op,
        scalar_name='y')
        z = scale_by_y(input1)
        w = scale_by_y(input2)
        # Creates a template that reuses the variables above.
        with tf.compat.v1.variable_scope(vs, reuse=True):
            scale_by_y2 = tf.compat.v1.make_template('scale_by_y', my_op,
            scalar_name='y')
            z2 = scale_by_y2(input1)
            w2 = scale_by_y2(input2)

```

Documentation

Given an arbitrary function, wrap it so that it does variable sharing.

Migrate to TF2

`tf.compat.v1.make_template` is a legacy API that is only compatible with eager execution enabled and `tf.function` if you combine it with `tf.compat.v1.keras.utils.track_tf1_style_variables`. See the model mapping migration guide section on `make_template` for more info:

https://www.tensorflow.org/guide/migrate/model_mapping#using_tfcompatv1make_te

Even if you use legacy apis for `variable_scope`-based variable reuse, we recommend using `tf.compat.v1.keras.utils.track_tf1_style_variables` directly and not using `tf.compat.v1.make_template`, as it interoperates with eager execution in a simpler and more predictable fashion than `make_template`.

The TF2 API approach would be tracking your variables using `tf.Modules` or Keras layers and models rather than relying on `make_template`.

Description

Used in the notebooks

- Use TF1.x models in TF2 workflows
- Linear Mixed-Effect Regression in {TF Probability, R, Stan}

This wraps `func_` in a `Template` and partially evaluates it. `Templates` are functions that create variables the first time they are called and reuse them thereafter. In order for `func_` to be compatible with a `Template` it must have the following properties:

- The function should create all trainable variables and any variables that

should be reused by calling `tf.compat.v1.get_variable`. If a trainable variable is

created using `tf.Variable`, then a `ValueError` will be thrown. Variables that are intended to be locals can be created by specifying `tf.Variable(..., trainable=False)`.

- The function may use variable scopes and other templates internally to

create and reuse variables, but it shouldn't use

`tf.compat.v1.global_variables` to

capture variables that are defined outside of the scope of the function.

- Internal scopes and variable names should not depend on any arguments that

are not supplied to `make_template`. In general you will get a `ValueError` telling you that you are trying to reuse a variable that doesn't exist if you make a mistake.

In the following example, both `z` and `w` will be scaled by the same `y`. It is important to note that if we didn't assign `scalar_name` and used a different name for `z` and `w` that a `ValueError` would be thrown because it couldn't reuse the variable.

As a safe-guard, the returned function will raise a `ValueError` after the first call if trainable variables are created by calling `tf.Variable`.

If all of these are true, then 2 properties are enforced by the template:

- Calling the same template multiple times will share all non-local

variables.

- Two different templates are guaranteed to be unique, unless you reenter the

same variable scope as the initial definition of a template and redefine it. An examples of this exception:

Depending on the value of `create_scope_now_`, the full variable scope may be captured either at the time of first call or at the time of construction. If this option is set to `True`, then all Tensors created by repeated calls to the template will have an extra trailing `_N+1` to their name, as the first time the scope is entered in the Template constructor no Tensors are created.

Returns

Raises